Software Design Analysis Report

Leave Management System | Planned system design

Project status: Planned - requirements and design stage. The documentation set (elicitation, SRS, UML models) is complete; the application has not yet been implemented.

A. Project Details

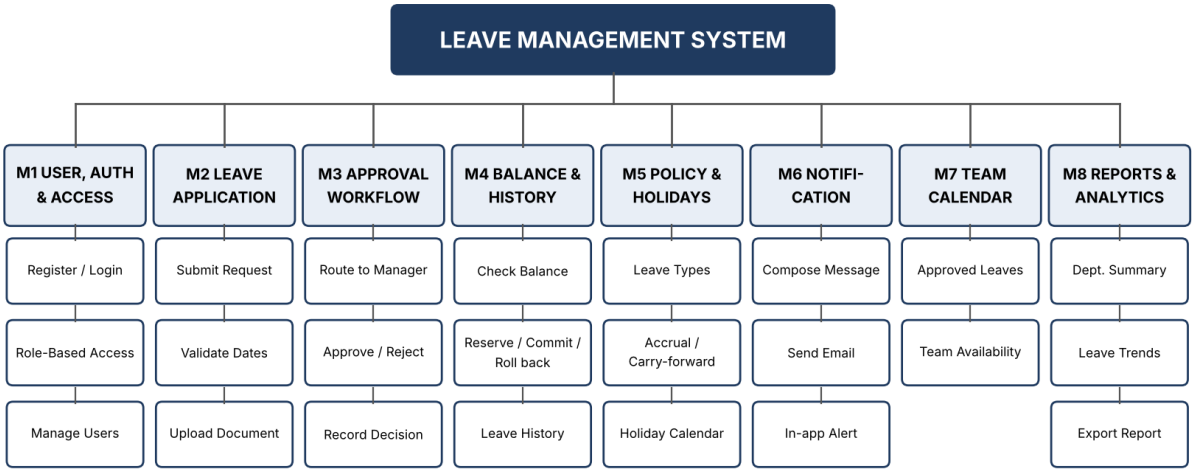
Project title	Leave Management System (LMS) - An Automated Web-Based Solution for Employee Leave Management
Team / roll numbers	Varun Kushwah - 2400290120275 (Team Lead / Backend) Vaibhav Gupta - 2400290120270 (Frontend) Tanya Bhadana - 2400290120251 (Database Design & Testing) Sudiksha Chauhan - 2400290120260 (Documentation & UI/UX Design)
Project objective	To digitize and automate the complete leave lifecycle of a single organization: employees apply for leave online, managers approve or reject requests, and the system automatically maintains accurate leave balances, history, notifications, a team calendar and HR reports, replacing paper forms and spreadsheets.
Technology stack	React.js with Tailwind CSS (frontend); Spring Boot (Java) REST API with Spring Security (backend); PostgreSQL / MySQL relational database; JWT and bcrypt for authentication and password hashing; SMTP email via Spring Boot Mail; Docker and Git/GitHub. The database choice (PostgreSQL or MySQL) will be confirmed during implementation.

B. Module Identification and Modularization

Total number of major functional modules: 8. The system follows the 3-tier architecture in the repository (presentation, application, data layer) and is divided by responsibility, so identity, leave submission, approval, balances, policy configuration, notifications, calendar and reporting can each be designed, tested and changed separately. This also matches the SRS requirement for a modular architecture (NFR-07). The relational database is shared storage, not an additional functional module.

ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	User, Authentication & Access	Register/login users, hash passwords, enforce role-based access (Employee, Manager, Admin), manage users.	Credentials / user data → session token, user ID, role.	Database (user records).
M2	Leave Application	Create and validate leave requests with type, dates, reason and optional supporting document.	Type, dates, reason, document → validated application.	M1 identity; M4 balance check; M5 leave types.
M3	Approval Workflow	Route requests to the manager, record approve/reject (with remarks) and drive status changes.	Application ID, decision, remarks → updated status.	M1 role; M2; M4; M6.
M4	Leave Balance & History	Calculate balance, reserve provisional days, commit or roll back deductions, keep leave history.	Employee ID, type, days → balance status, history.	M5 policy rules; Database.
M5	Policy & Holiday Configuration	Admin maintains leave types, accrual rate, carry-forward limits, holidays and departments.	Policy / holiday data → saved configuration.	M1 (admin role); Database.
M6	Notification	Compose and send email and in-app alerts on submission, approval and rejection.	Event type, recipient, message → delivery status.	External SMTP server.
M7	Team Calendar	Show approved leaves and holidays per team/department for workforce planning.	Department, month → calendar entries.	M3 approved leaves; M5 holidays.
M8	Reports & Analytics	Produce department summaries and leave-trend reports, with export.	Period, department → report / export file.	M4 history; M5; Database.

Module hierarchy / decomposition diagram



The chart shows the system decomposed into eight modules with their main subfunctions (derived from the use case, activity and class diagrams in the repository).

C. Design Strategy

Selected strategy: TOP-DOWN

- The project began as a complete problem statement (a centralized digital leave system) and was refined step by step: requirements elicitation → SRS → architecture → modules → classes.
- Because it is a planned, documentation-first project, requirements and scope were fixed before any code, so decomposing from the whole system downward is the natural fit.
- Each module has clear boundaries, so the four team members can be assigned modules (backend, frontend, database, documentation) and test them independently before integration.
- Bottom-up reuse is limited here: there is no existing component library to assemble, and the leave workflow is driven by the end-to-end business process.

Evidence: how the design moved from the whole system to modules.

Level	Description	Example from this project
1. System	Whole LMS and its actors	Employee, Manager, Administrator/HR; goal: automate leave requests and approvals.
2. Layers	3-tier architecture (system architecture diagram)	Presentation, Application (REST API), Data layer, SMTP integration.
3. Modules	Eight responsibilities in the Application layer	M1 Auth … M8 Reports (section B).
4. Subfunctions	Tasks inside each module (use cases)	Apply for leave, check balance, approve/reject, send notification, generate report.
5. Classes / data	Class diagram and database entities	User, Employee, Manager, Administrator, LeaveApplication, LeaveBalance, LeaveType, Notification, LeavePolicy, Holiday, Report.

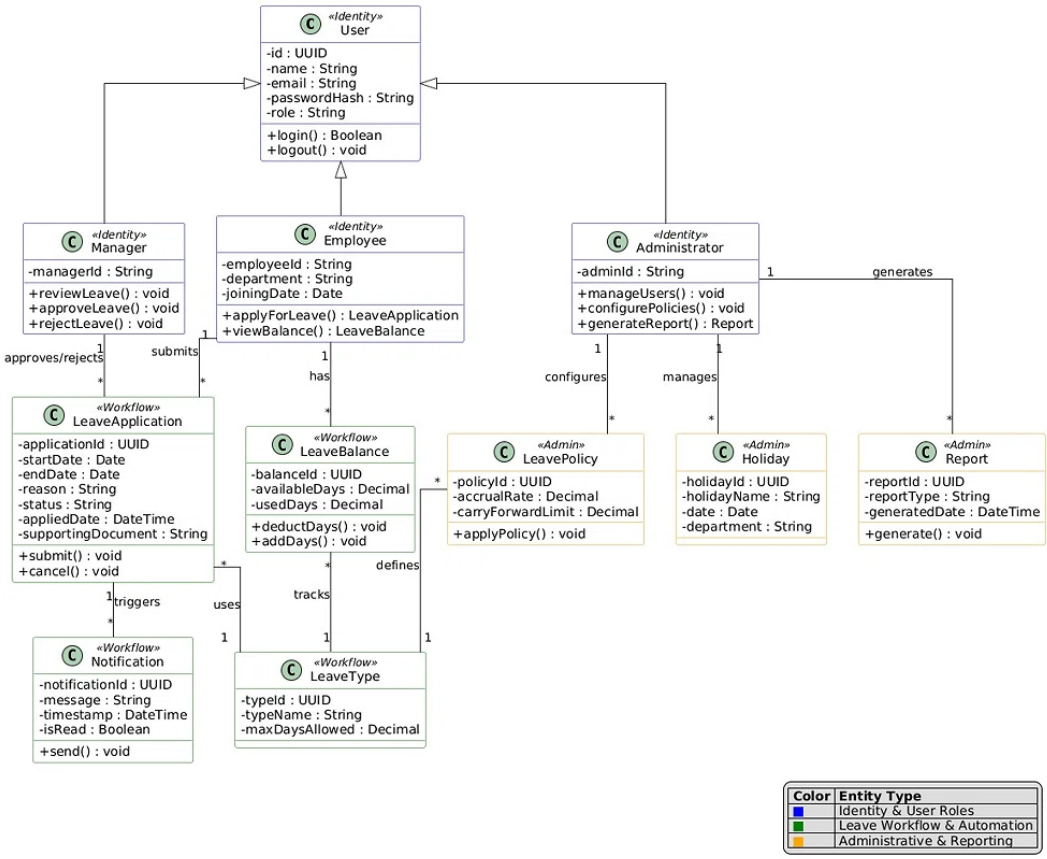
D. Design Approach

Selected approach: OBJECT-ORIENTED

- The repository design is a UML class diagram: entities such as **User**, **Employee**, **Manager**, **Administrator**, **LeaveApplication**, **LeaveBalance**, **LeaveType**, **Notification**, **LeavePolicy**, **Holiday** and **Report** carry their own data and behaviour.
- Real-world objects map directly to classes (an employee submits a leave application; a manager approves it; a balance deducts days), which keeps the design close to the domain.
- Inheritance is used naturally: Employee, Manager and Administrator specialize User, sharing id, name, email, passwordHash, role, login() and logout().

- Encapsulation supports security and integrity: LeaveBalance hides its available and used days behind deductDays() / addDays(); status changes go through submit() / cancel() and approveLeave() / rejectLeave().
- New leave types or policies are added as data (LeaveType, LeavePolicy) rather than new code, supporting maintainability (NFR-06, NFR-07).

Evidence: class diagram from the project repository (docs/images/class-diagram.png).



Mapping of classes to modules: M1 → User, Employee, Manager, Administrator; M2 → LeaveApplication; M3 → LeaveApplication status workflow, Manager; M4 → LeaveBalance; M5 → LeaveType, LeavePolicy, Holiday; M6 → Notification; M7 → approved LeaveApplication and Holiday data; M8 → Report.

E. Coupling Analysis

The goal is low coupling: modules exchange only the data needed for a task and never depend on another module's internal data or implementation. Interactions are proposed at design stage and will be confirmed when API routes and schemas are implemented.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M1 Auth ↔ M2 / M3 / M5 (protected features)	User ID and role from a validated JWT token, used to authorize each request.	Data coupling; low (only ID and role are passed).	Use a shared Spring Security filter for authorization; never pass passwords or full user objects.
M2 Application → M4 Balance	Employee ID, leave type and number of days; returns sufficient / insufficient.	Data coupling; low.	Expose one narrow interface, checkBalance(empId, type, days); M2 does not read balance tables directly.
M2 Application → M3 Workflow	The submitted leave application passed for routing to the manager.	Stamp coupling if the whole LeaveApplication object is passed; moderate.	Pass only applicationId and let M3 load what it needs through a repository interface.
M3 Workflow → M4 Balance	Decision outcome: commit the deduction on approval, release the reservation on rejection.	Control coupling if an approve/reject flag selects the behaviour; moderate.	Provide two explicit operations, commitDeduction() and releaseReservation(), instead of a flag.
M3 Workflow → M6 Notification	Event type, recipient ID and message text.	Data coupling; low.	Publish an event; M6 builds and sends the email, so M3 does not know SMTP details.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M7 Calendar / M8 Reports → M3, M4, M5	Read-only queries for approved leaves, history and holidays.	Data coupling; low-to-moderate if raw records are shared.	Use summary/query methods returning only needed fields; reports never modify leave data.
React UI ↔ Spring Boot API	JSON request and response payloads for each feature.	Data coupling through API messages.	Stable REST endpoints, validated payloads, all database access kept in the backend.

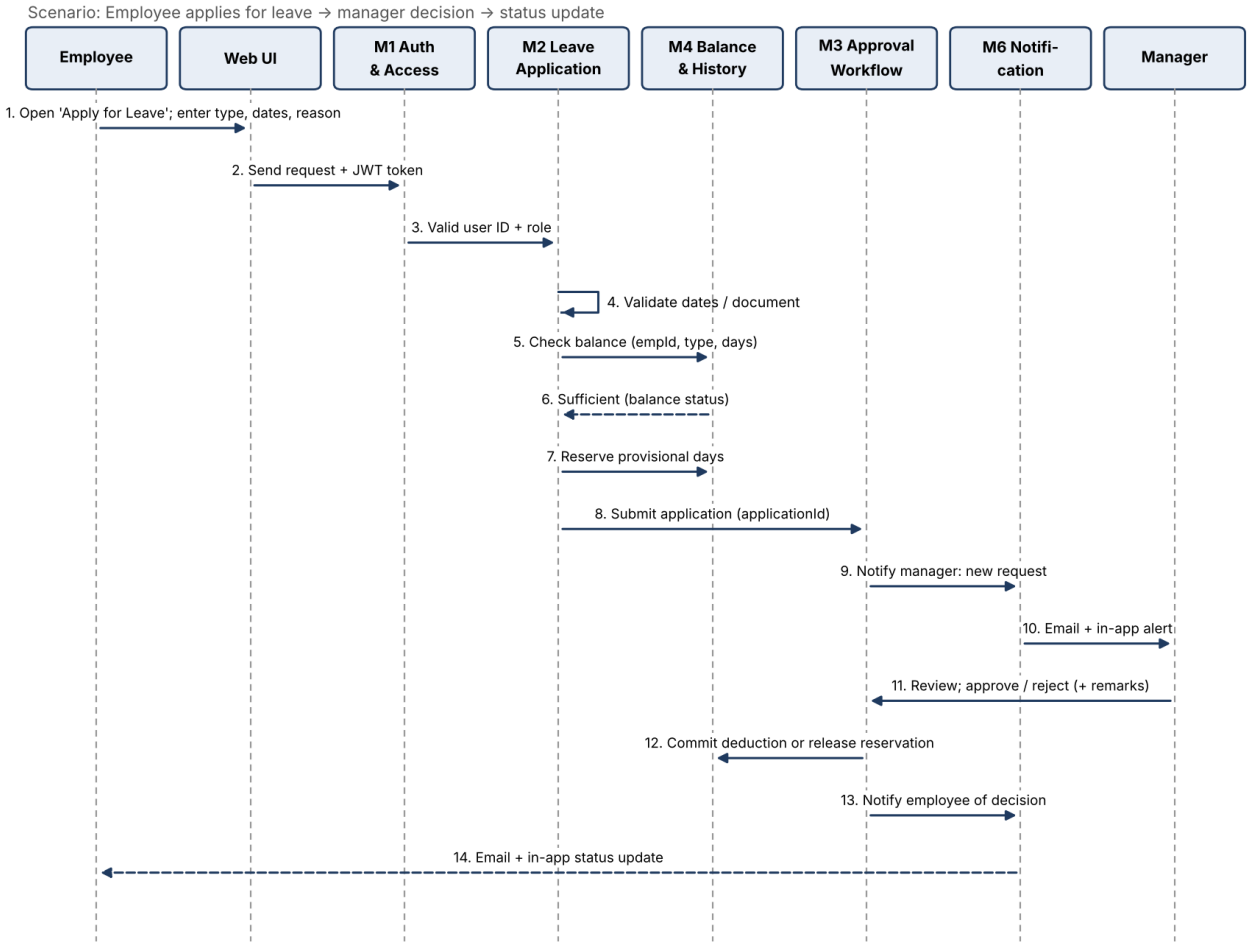
F. Cohesion Analysis

High cohesion means every task inside a module contributes to its single main responsibility. The analysis below reflects the planned module boundaries.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M1 User, Authentication & Access	Identify users and control what each role may access.	Functional cohesion; high.	Keep hashing, login and RBAC checks here; do not add leave calculations.
M2 Leave Application	Create and validate a leave request.	Functional cohesion; high.	Leave balance rules stay in M4; M2 only calls it.
M3 Approval Workflow	Move a request through review to approved or rejected.	Functional cohesion; high.	Keep status transitions here; delegate balance updates and messages to M4 and M6.
M4 Leave Balance & History	Maintain the accuracy of balances and leave records.	Functional cohesion; high.	Keep reserve, commit, rollback and history together in one transaction boundary.
M5 Policy & Holiday Configuration	Store the rules (types, accruals, carry-forward, holidays).	Functional cohesion; high.	Hold configuration only; enforcing rules is done by M2 and M4 reading it.
M6 Notification	Deliver messages about workflow events.	Functional cohesion; high.	Keep message templates and delivery here; no approval logic.
M7 Team Calendar	Display approved leaves and holidays by team.	Functional cohesion; high.	Read-only; avoid mixing in report calculations.
M8 Reports & Analytics	Summarize leave data into reports and exports.	Functional cohesion; high (sequential steps: query → aggregate → export).	Read data through defined queries; avoid editing leave records.

G. Module Interaction and Design Quality

Sequence diagram - planned request flow (employee applies for leave, manager decides, employee is notified). The diagram is conceptual; API paths and storage details may change during implementation.



Interaction summary

- The employee works through the web UI; every request carries a token that M1 verifies before the call reaches M2.
- M2 validates the request, asks M4 for the balance, and on success M4 reserves the days provisionally before the application is routed to M3.
- M3 holds the request in the manager's pending queue; M6 alerts the manager by email and in-app message.
- On approval M4 commits the deduction and the record appears in the team calendar; on rejection M4 releases the reservation. M6 then informs the employee.

Design quality summary

Quality goal	How the proposed design supports it
Maintainability	Each major function sits in its own module; policies and holidays are data, so changes do not require redesign.
Testability	Validation, balance calculation, status transitions, notifications and report totals can be tested independently.
Low coupling / high cohesion	Modules exchange narrow values (IDs, days, status) and keep unrelated responsibilities separate; all modules show functional cohesion.
Security	Proposed JWT authentication, bcrypt password hashing, role-based access control and input validation; to be reviewed and tested at implementation.
Data integrity	Provisional reservation with commit/rollback inside database transactions prevents overdrawn or double-booked balances.

Items to confirm during implementation: final database choice (PostgreSQL or MySQL), authentication/session approach, REST endpoint design, database schemas, and notification delivery method. This report does not claim that any feature is already built because the project is still at the design stage.